

系统架构设计

本文参考智元远征A2人形机器人旗舰款、宇树UnitreeG1人形机器人、宇树机器狗Go2、南华大学机器人智能底盘、幻尔智能机器人。

系统整体采用分层模块化设计，云端管理层、感知决策层、实时运控层、机电执行层。

已有硬件

幻尔小车机架（含编码器电机*4）、jetson nano、rk3568 4+64GB、2D激光雷达、深度相机、麦克风阵列、EtherCAT伺服电机

软件移植

大脑与小脑均采用docker进行开发，方便一整套直接打包移植，应用层->话题接口层->驱动层，移植后只需要修改驱动层即可

驱动层全用Lifecycle，算法层和应用层只用普通Node。

优点如下

1. 软件上与硬件完全隔离，可移植到任何性能足够的机器上
2. 可以按照自己的节奏走，完全不受限于他人
3. 收获个人机器人项目写简历上、完成企业实习写简历上、个人项目后续还可以用来给学校打比赛继承下去

一举四得，非常nice。

目标如下

使用现有设备完成所有功能，必须确保极高的移植性、扩展性、稳定性，完成后移植到公司底盘上。

核心问题

jetson nano-交换机-rk3568+电机通信-stm32控制器

更新移植：jetson orin nx - 交换机 - rk3588+电机通信 - 控制器

目前小车是一体化的，大号的rk3568和交换机完全塞不进，但是stm32f4控制器有源码、原理图、通信协议解析。

方案一：

使用rk3568核心板+自画双网口底板，不使用交换机（或者微型交换机），优点是符合物理架构，性能足够。但是耗时太久了。

方案二：

采用软件模拟方式

物理上使用jetson nano-stm32控制器

软件上【大脑docker】-虚拟以太网段-【小脑docker】-【控制器】

对应 大脑docker - 交换机 - 小脑docker - 控制器

大脑docker：原本就是这样配置的

虚拟以太网段：完全能够模拟真实交换机功能

小脑docker：原本计划小脑ros2就是使用docker的

控制器：可以是串口、EtherCAT、CAN都没问题

主要的应用部分均采用docker进行开发，移植是极其方便的，关于以太网和电机控制也只需要更改驱动接口即可。考虑到时间关系和移植方便性等等因素，我们选择方案二来解决问题。

大脑docker运行Ubuntu22.04无桌面版+ROS2 Humble，跑MQTT+WebSocket+WebRTC通信、YOLO+OpenCV视觉（YOLOv8n + TensorRT + 低分辨率 + 5Hz推理）、2DSLAM、RTK、系统监控、nav2、网络API语音转文本、网络API文本转语音、网络API LLM/VLM、任务树、PTP Master

小脑docker运行Ubuntu22.04无桌面版+ROS2 Humble（不使用PREEMPT-RT），跑MQTT/TCP通信、电机通信控制（与stm32控制器串口通信）、IMU姿态平衡、odom里程计、twist话题控制、BMS通信监控（与stm32控制器串口通信）。

使用jetson nano, 4GB+64GB

当性能出现瓶颈时使用

```
{  
  大脑docker cpu0+1 2.5GB  
  小脑docker cpu2 0.5GB  
  宿主机+系统 剩余使用  
}
```

刷机为ubuntu18的无桌面版，需要rviz2可以使用x11转发来做。(gazebo基本用不上)

连接拓扑

 优化-拓扑.drawio (1)

云端管理层

该层为面向管理人员的服务端，管理人员可轻松使用平台进行相关控制管理相关操作。

接口协议选择

协议	一句话定位	类比	消息
MQTT	轻量级“消息快递员”	发短信（小数据、低功耗、可丢包）	控制指令和状态上报（轻量可靠）
WebSocket	双向“实时管道”	打电话（长连接、低延迟、双向通信）	信令交换和实时数据推送（双向低延迟）
WebRTC	“视频流高速公路”	看电视（大数据、高带宽、连续流）	视频流、音频流、点云数据（超低延迟抗丢包）

应用功能

服务端（C#/.NET + WebRTC + MQTT）

需求	技术栈
MQTT Broker（本地）	EMQX（或 Mosquitto），但 .NET 可用 <code>MQTTnet</code> 实现轻量级 Broker（可选）
MQTT 客户端（后端）	<code>MQTTnet</code> （连接 Broker，订阅/发布消息）
WebRTC 信令服务器	ASP.NET Core + WebSocket（或用 SignalR 作为信令通道）
WebRTC 媒体服务器	Janus 或 SRS （支持 WebRTC），.NET 通过 HTTP API 控制房间/推流
实时视频流处理	前端（浏览器/App）使用 WebRTC 直接推流/拉流，.NET 不直接处理媒体流
后端框架	ASP.NET Core 8.0（WebAPI + SignalR + WebSocket）
数据存储	PostgreSQL + EF Core（关系数据） + InfluxDB（时序数据） + MinIO（视频截图）
设备/用户认证	JWT + ASP.NET Core Identity
容器化部署	Docker + Docker Compose（包含 EMQX、Janus、数据库、.NET 服务）

1. 设备连接与鉴权管理

功能	说明
设备注册	机器人首次上线时向服务器注册设备ID
设备认证	JWT Token认证，确保只有合法设备能接入
在线状态管理	维护设备在线/离线状态，支持 <code>Last will</code> 断线通知
心跳保活	检测设备是否在线，超时自动标记离线

2. 实时数据接入与存储

功能	说明
MQTT Broker	接收机器人上报的状态数据（位置、电量、速度、温度等）
时序数据存储	将传感器数据存入InfluxDB，支持历史查询
关系数据存储	设备信息、用户信息、巡检记录存入SQL
视频/点云存储	通过MinIO存储关键帧截图、点云文件（可选）

3. WebRTC媒体服务管理

功能	说明
SFU房间管理	通过HTTP API控制Janus创建/销毁房间
推流/拉流权限控制	控制哪些客户端可以推流/拉流
信令转发	转发WebRTC SDP/ICE信令（通过SignalR或WebSocket）

4. 远程控制指令下发

功能	说明
指令发布	通过MQTT向机器人下发控制指令（开始巡检/返航/紧急停止等）
指令确认	接收机器人对指令的执行确认，超时重发
任务调度	支持定时巡检任务配置，到时间自动下发

5. 巡检任务管理

功能	说明
任务创建	管理人员创建巡检任务（路线、巡检点、检测项）
任务下发	将任务下发到指定机器人
任务状态跟踪	实时跟踪任务执行进度（进行中/完成/异常中断）
任务历史	查询历史巡检任务记录和结果

6. 异常报警与事件管理

功能	说明
报警规则配置	配置触发报警的阈值（电量<20%、温度>60℃等）
实时报警推送	通过MQTT/WebSocket向客户端推送报警信息
报警历史	存储和查询历史报警记录

7. 数字孪生数据接口

功能	说明
实时状态推送	通过WebSocket向客户端推送机器人实时状态（位置/姿态/传感器）
历史轨迹回放	提供历史轨迹数据用于回放

8. 用户与权限管理

功能	说明
用户认证	登录/登出/JWT Token签发与刷新
角色权限	RBAC权限控制（管理员/操作员/只读用户）
操作日志	记录用户操作日志（谁在什么时间做了什么）

客户端(Nuxt + WebRTC + MQTT)

主要框架: Nuxt 4 + Vue 3 + TypeScript + Vite + pnpm

- UI: Element Plus(或 Naive UI)+ UnoCSS + Iconify
- 状态/工具: Pinia + VueUse
- 实时通信: MQTT.js + WebRTC + janus.js + SignalR 客户端
- 3D/孪生: Three.js + TresJS + URDF Loader(点云用 Potree)
- 可视化: ECharts(vue-echarts),地图用 Cesium / Mapbox
- 工程化: ESLint + Prettier + Vitest + Playwright + Docker + Nginx

1. 实时监控（核心功能）

功能	说明
视频实时预览	通过WebRTC拉取机器人摄像头视频流
点云可视化	通过WebRTC拉取激光雷达点云数据，用Three.js/Potree 3D渲染
传感器仪表盘	实时显示电量、速度、温度、IMU姿态等数据（ECharts图表）
机器人位置地图	在2D/3D地图上显示机器人当前位置、航向、轨迹
设备状态指示	在线/离线/异常状态指示灯

2. 远程控制

功能	说明
手动控制面板	虚拟摇杆/键盘控制机器人前进/后退/转向（通过MQTT下发指令）
一键巡检	点击按钮下发“开始巡检”任务
一键返航	点击按钮让机器人返回充电桩/起点
紧急停止	红色急停按钮，立即下发停止指令（P0级高优先级）

3. 巡检任务管理

功能	说明
任务配置	在地图上设置巡检点、规划巡检路线
任务下发	选择任务并下发到指定机器人
任务进度可视化	实时显示当前任务执行进度和机器人位置
任务历史查询	查看历史任务列表、详情、执行结果

4. 报警与事件管理

功能	说明
实时报警弹窗	收到报警时弹窗提示并语音播报
报警列表	展示当前未处理的报警列表（高/中/低优先级）
报警确认	操作员确认报警，记录处理人/时间/备注
报警历史查询	查看历史报警记录

5. 数据可视化与分析

功能	说明
传感器时序图	用ECharts显示温度/电量/速度等参数的时序曲线
轨迹回放	回放机器人历史运动轨迹（2D地图+3D孪生）
巡检报告生成	根据巡检结果生成报告（PDF/Excel导出）

6. 数字孪生3D展示

功能	说明
3D机器人模型	用Three.js加载URDF模型，同步显示机器人实时姿态
实时数据驱动	模型位置/关节角度跟随真实机器人运动
传感器数据叠加	在3D场景中显示激光点云、检测框

7. 语音交互

功能	说明
语音对讲	通过WebRTC实现客户端与机器人的双向语音通话
语音指令	通过语音识别生成控制指令（“去A点巡检”等）

8. 系统管理（管理员）

应用层

使用接口层+算法层构建一个个应用APP，如网关系统、视觉系统、音频系统、导航系统应用功能。

算法层各司其职完成了核心任务链路的整合。应用层主要做的是顶层任务 + 决策 + 监控。

二开层

将功能节点封装为RPC接口，即使用http作为通信方式用于二次开发，屏蔽了开发语言的差异性，任何支持http的语言都可以进行二次开发。

任务层

该层以任务为单位，无论是应用层任务还是二开层任务，都需要放入该层注册，由系统状态机、行为树进行任务编排调度，是全局大脑。

同时可接入大模型（二次开发者）作为拟人角色使用RPC接口发布任务参与编排。如注册了电量低返航充电任务、巡航任务、检测任务，该层会执行巡航、中途调用检测、电量低时触发返航充电。它不再关注底层的“怎么动”（那是应用层的事），而是关注“做什么”和“按什么顺序做”。

该层由任务注册模块、任务编排模块（行为树BehaviorTree.CPP、py_trees）、调度仲裁模块构成。

应用功能

系统	核心功能
通信系统	WIFI/5G - 加解密 - MQTT/WebSocket/WebRTC驱动 - 接口话题 - 实现指令接收与状态上报
传感系统	视觉（目标识别、深度检测）、音频（异响匹配）、建图（2D/3D SLAM）、定位（RTK/融合/里程计）、安全（BMS/电机/系统状态监控）...
控制系统	Nav2导航（发布 <code>/navigate_to_pose</code> 目标），并具备急停与遥控接管机制
交互系统	语音（语音转文字/文字转语音）、网络（传感与控制数据通道）
AI系统	在线/离线大模型（LLM/VLM），通过Function Calling调用交互与任务系统
任务系统	任务注册（支持动态加载）、任务编排（行为树/状态机）、任务调度（基于优先级或事件）
配置系统	全局配置文件管理，支持环境切换与热加载

实时运控层

该层作为整机小脑部分，负责电机控制、imu姿态平衡、odom里程计、nav2 twist话题控制、硬件安全系统(电机+BMS)。采用驱动-节点接口-应用分层，Docker部署方便一键打包移植并隔离环境。

基础配置

- 1. 主控：jetson nano 分出核心/ RK3568 4GB+64GB+1Tops 升级-> RK3588
- 2. 传感器：IMU、BMS
- 3. 软件：Ubuntu22.04 Server无桌面版+ ROS2 Humble + （使用EtherCAT才加入PREEMPT-RT）

软件架构

公共层

该层作为贯穿所有层的隐形支柱，负责安全、诊断、日志、配置、全局时间同步等公共功能。独立层

驱动层

常规Linux驱动编写方式，与ros无关。独立层

系统服务层

一些系统服务，与ros无关，如AP配网、PTP时间同步服务器。独立层

驱动接口层

由驱动文件加载+接口组成，用于隔离软硬件，方便移植。比如电机驱动板-串口-接口话题 /motor，电机-EtherCAT-接口话题 /motor

接口话题不变，更改驱动即可。

算法层

面向对象思想的独立算法模块，可配置数据源，方便复用。如pid模块、ik模块、ki模块、odom模块、twist模块等。

应用层

算法层各司其职完成了核心任务链路的整合。应用层主要做的是顶层任务 + 决策 + 监控。

应用功能

电机控制、姿态平衡与指令输出（接收大脑twist/cmd_vel消息+imu补偿->最终指令）、里程计、安全监控

机电执行层

该层是物理接口，接入控制器，如CAN、EtherCAT、串口控制器等